

Archivist MVP: PRD, Stack and Build Path (v2)

Constraints: MVP due tomorrow 9am. Next.js frontend. Deployed with a live link.

Hindsight memory must be central. **Strategy:** one Next.js app on Vercel, precomputed synthetic data, deterministic maths for outcomes, LLM only for diagnosis and brief writing. Everything below is scoped for one overnight push.

1. Product in one paragraph

Archivist audits a brand's published content archive, flags decaying pages, recommends a refresh type for each, and then *measures whether the refresh worked* using a difference-in-differences comparison against untouched pages in the same cluster. Every measured outcome is retained in Hindsight, and the next brief recalls those outcomes so recommendations get better. Week 1 advice is generic; week 5 advice is specific to this brand.

2. MVP scope (hard limits)

In

- One fictional B2B SaaS brand, about 40 posts, 4 topic clusters.
- Three refresh types: `update_stats`, `add_faq`, `rewrite_intro`.
- Decay detection (rules), cause diagnosis (LLM), weekly brief (LLM + memory).
- "Apply refresh" action, "Advance 4 weeks" button, DiD outcome calculation.
- Hindsight retain (outcomes, facts) and recall (before each brief).
- Memory panel showing what the agent has learned.
- Week 1 vs week 5 comparison view.

Out (for tomorrow)

- CSV upload, auth, multi-user, real analytics integrations, topic-gap detection beyond a simple coverage table, the reflect step, a database.

3. Key design trick: counterfactual series

The data generator makes **two trajectories per candidate page**: one if refreshed (with a planted effect for its refresh type) and one if left alone. The app reveals the trajectory matching the user's action. This gives you a real before/after with a known answer, no backend simulation, and no database.

4. Outcome measurement (DiD)

```
effect = (page_after / page_before - 1)
        - median over controls of (control_after / control_before - 1)
```

- Windows: 8 weeks pre, 4 weeks post (demo speed), skip the first week after the refresh.
- Controls: untouched pages in the same cluster, similar traffic band, at least 3 or return `inconclusive`.
- Threshold fixed in config before running: `improved` if effect > +15 points, `no_effect` between -5 and +15, `worse` below -5.
- Store effect size, window, control count and confidence, not just the label.

5. Planted patterns (ground truth)

Case	Planted truth	What the agent must say
3 pages, cause outdated stats, <code>update_stats</code>	+30 to +45 points	improved
2 pages, <code>add_faq</code>	about 0 points	no_effect
1 page that rebounds on its own	control pages rebound too	no_effect (not credited to the refresh)
1 page with a contaminated control	one control was also touched	flagged, control excluded
Missing topic in a cluster	no post covers it	listed as a gap

Keep `groundtruth.json` and check the agent against it. This is your proof in the demo.

6. Tech stack

Layer	Choice	Why
App	Next.js (App Router) + TypeScript	One project for UI and API routes, one deploy
Styling	Tailwind CSS + shadcn/ui	Fast, polished look
Charts	Recharts	Before/after and control-vs-page lines
LLM	Groq via the OpenAI-compatible SDK	Fast; use <code>openai/gpt-oss-120b</code> or <code>qwen/qwen3-32b</code>
LLM output	JSON mode + Zod validation + retry, no tool calling	Avoids function-calling errors
Memory	Hindsight Cloud	Required; use the HTTP API or the JS client if available
Data	Static JSON files in the repo	No database to set up
Hosting	Vercel	One-click deploy from GitHub
Coding help	Any coding agent from the hackathon PDF	Speeds up boilerplate

If the Hindsight JS client doesn't work, call its REST API directly with `fetch`. Verify the exact endpoints in the docs during the spike.

7. Repo structure

```
archivist/  
  data/  
    posts.json          # 40 posts: id, title, cluster, summary, published,  
cause hints  
    metrics.json        # weekly views per post (history + both trajectories)  
    groundtruth.json  
  scripts/  
    generate-data.ts    # deterministic generator with planted patterns  
  src/  
    app/  
      page.tsx          # dashboard: archive, decay list, brief  
      api/brief/route.ts # detect + recall + LLM brief  
      api/action/route.ts # user applies a refresh  
      api/advance/route.ts # advance weeks, compute DiD, retain outcome  
      api/memory/route.ts # recall for the memory panel  
    lib/  
      did.ts            # difference-in-differences maths  
      decay.ts          # rule-based decay detection  
      hindsight.ts      # retain / recall wrappers  
      llm.ts            # Groq calls + Zod schemas + retries  
    components/         # BriefCard, MemoryPanel, OutcomeChart, WeekSwitcher  
  README.md
```

8. App flow (what the demo shows)

1. Load dashboard at week 1: decayed pages listed with drop from peak.
2. Click "Generate brief": memory is empty, so recommendations are generic.
3. Apply refreshes: `update_stats` on 3 pages, `add_faq` on 2.
4. Click "Advance 4 weeks": the app reveals the chosen trajectories.
5. DiD runs per refreshed page; outcomes appear with effect size and control count.
6. Each outcome is retained in Hindsight as a sentence with numbers.
7. Generate the week 5 brief: it recalls outcomes and now says stat refreshes work here and FAQ additions do not, and ranks new decayed pages accordingly.
8. Show the memory panel and the week 1 vs week 5 comparison.

9. Hindsight usage

- **Retain:** content facts (page topic, format), diagnosed causes, refresh actions, and DiD outcomes written as clear sentences, e.g. "Refresh type `update_stats` on a 'pricing' cluster page improved views by 38 points vs controls (5 controls, 4 weeks)."
- **Recall:** before each brief, query per candidate, e.g. "outcomes of refresh types for pages decayed due to outdated stats." Pass results into the brief prompt and show

them as evidence on the card.

- **Bank:** one bank for the brand.
- The README must explain this clearly, since memory is 25% of judging.

10. Overnight build plan

Time-boxed. If a block runs over, use the cut list.

Block	Task	Done when
0:00 to 0:45	Create Next.js app, install Tailwind/shadcn/Recharts, get Groq and Hindsight keys, spike: retain one sentence and recall it	Recall returns your sentence
0:45 to 2:15	Write <code>generate-data.ts</code> with planted patterns and counterfactual series; generate the JSON	Charts of a few pages look realistic
2:15 to 3:15	<code>did.ts</code> and <code>decay.ts</code> ; test <code>did.ts</code> against groundtruth	The five planted cases give the expected labels
3:15 to 4:45	LLM diagnosis and brief routes with Zod validation; Hindsight retain/recall wrappers	Brief returns valid JSON with evidence field
4:45 to 7:00	Dashboard UI: decay list, brief cards, action buttons, advance button, outcome chart	Full loop works locally
7:00 to 8:00	Memory panel and week 1 vs week 5 view	Week 5 brief visibly differs
8:00 to 9:00	Deploy to Vercel, set env vars, test live	Live link completes the full loop
9:00 to 10:00	README, record demo video, submit	Submission complete

Cut list, in order: memory panel polish, week comparison view, gap table, charts (use plain numbers). **Never cut:** DiD outcome, Hindsight retain and recall, deployed link.

11. Risks and mitigations

- **Hindsight API surprises:** do the spike first; keep wrappers thin.
- **LLM invalid JSON:** Zod validation, one retry, then fallback to a template brief.
- **Vercel serverless has no persistent disk:** keep week and action state in the client (React state), and keep durable learnings in Hindsight.
- **Demo looks scripted:** show the ground-truth file and the raw DiD numbers so judges see it is measured.

- **Timeouts on Vercel:** keep each LLM call small; brief generation should take under 10 seconds.

12. Submission checklist

- ☐ Public GitHub repo with README (problem, architecture, how Hindsight is used)
- ☐ Live Vercel link that runs the full loop
- ☐ Short demo video (about 60 seconds, follow section 8)
- ☐ Explanation of Hindsight memory usage
- ☐ Check the PDF and content guide for when the article, social post and video are due

13. Checkpoints to think through before coding

1. In the DiD, what does a "similar traffic band" mean numerically, and how will you pick controls in code?
2. How will you phrase retained outcomes so recall finds them with a natural-language query?
3. What will your fallback brief say if the LLM fails, so the demo never breaks?
4. If Hindsight recall returns nothing, how does the UI make "memory empty" obvious in week 1?